

Camera Image Capture and Reconstruction

Objective:

To capture an image using the OV7675 camera connected to the Arduino Nano 33 BLE and reconstruct the captured image on a computer using Python. **(Note: Install Python, NumPy, Matplotlib)**

Procedure:

1. Connected the OV7675 camera module to the Arduino Nano 33 BLE board.
2. Opened the Arduino code for capturing an image frame
3. Uploaded the program to the Arduino Nano 33 BLE board.
4. Opened the Serial Monitor and executed the program.
5. The camera captured an image and printed the image data as hexadecimal pixel values (RGB565 format) in the Serial Monitor.
6. Copied the complete hexadecimal pixel data generated by the Arduino program.
7. Opened the Python image reconstruction program in Visual Studio Code.
8. Pasted the copied hexadecimal pixel values into the HEXADECIMAL_BYTES array of the Python program.
9. Executed the Python program.
10. The Python program converted the RGB565 hexadecimal pixel values into RGB color values.
11. The converted pixel values were arranged according to the camera resolution (176 × 144).
12. The reconstructed image was displayed using Matplotlib.
13. Verified that the displayed image matched the object captured by the camera.

Output:

```
Output  Serial Monitor  X
Message (Enter to send message to 'Arduino Nano 33 BLE' on 'COM4')

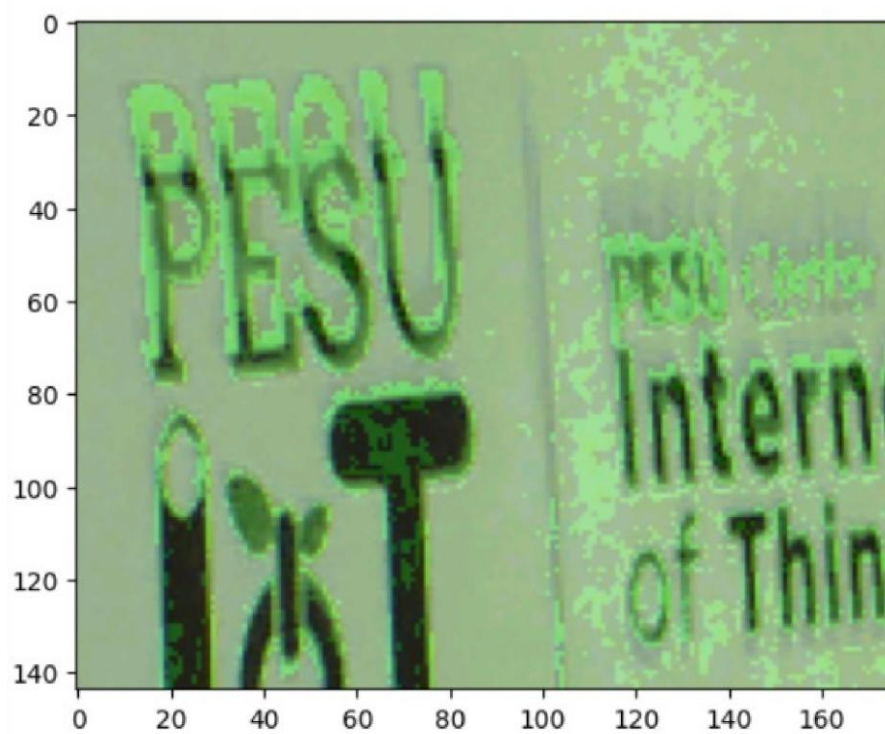
Starting Camera...
Camera OK
Pixels = 25344
HEXADECIMAL_BYTES = [
0xA95B, 0xAA5B, 0xA95B, 0xA953, 0xCA53, 0xE953, 0xA953, 0xCA39, 0x0529, 0x8322, 0xA42A, 0x0312, 0xA110, 0xA110, 0xA110, 0xA110,
0x8110, 0xA110, 0xC110, 0x6112, 0x0421, 0xC639, 0xC839, 0x0A3B, 0xC839, 0xC839, 0xA731, 0x6529, 0x6531, 0xE522, 0xE622, 0x8422,
0x641A, 0x441A, 0x231A, 0x231A, 0x4422, 0x842A, 0xC42A, 0x4631, 0x0629, 0x8639, 0xE541, 0x674B, 0x8A53, 0xCA5B, 0xA95B, 0xC95B,
0xC95B, 0xE95B, 0xE95B, 0xE95B, 0xC95B, 0xC95B, 0xCB5B, 0xCB5B, 0xCB5B, 0xEB5B, 0xCB5B, 0xC95B, 0xEB5B, 0xEB5B, 0xEB5B, 0xCB5B,
0xCB5B, 0xC95B, 0xC95B, 0xEB5B, 0xAB5B, 0xC95B, 0x0964, 0xEB5B, 0xC953, 0xC953, 0xAA53, 0xAA53, 0xA953, 0x694B, 0x4A43, 0x2843,
0x0843, 0xE839, 0x274B, 0xEB41, 0xC739, 0x0843, 0xA739, 0xA839, 0x4531, 0xC841, 0x2743, 0xA839, 0xE739, 0x484B, 0x0843, 0xC841,
0xC731, 0xC631, 0xE53B, 0xC541, 0xC73B, 0x0843, 0x0843, 0xC73B, 0x0743, 0xA843, 0xA843, 0xA84B, 0x654B, 0xA553, 0xC053, 0xC05B,
0xA55B, 0xA55B, 0xA55B, 0xA55B, 0xA55B, 0xA55B, 0xA55B, 0xA55B, 0xA55B, 0xA55B, 0xA55B, 0xA55B, 0xA55B, 0xA55B, 0xA55B, 0xA55B,
0xE32A, 0x8639, 0x2543, 0x884B, 0xEA5B, 0xCA5B, 0xEA5B, 0xCA5B, 0xE95B, 0xEA5B, 0xE95B, 0xE95B, 0xC95B, 0xE95B, 0xE95B, 0xE95B,
0xE95B, 0xC95B, 0xC95B, 0xC95B, 0xE95B, 0xC95B, 0xC95B, 0xC95B, 0xE963, 0xC963, 0xC95B, 0xC95B, 0xC95B, 0xEA5B, 0xC95B, 0xAA5B,
0x496C, 0x094B, 0x241A, 0x8310, 0x8110, 0x8110, 0x8110, 0x8110, 0x8110, 0x8110, 0x8110, 0x8110, 0x8110, 0x8110, 0x8110, 0x8110,
0x8110, 0x8110, 0x8110, 0x8110, 0x8110, 0x8110, 0x8110, 0x8110, 0x8110, 0x8110, 0x8110, 0x8110, 0x8110, 0x8110, 0x8110, 0x8110
]
Finished.
```

Successfully captured an image using the OV7675 camera and obtained 25,344 RGB565 pixel values in hexadecimal format

What it does:

- 1. Initializes the camera
- 2. Captures one image frame
- 3. Counts total pixels
- 4. Converts pixels to hexadecimal
- 5. Sends pixel data to Serial Monitor
- 6. Stops after capturing one frame

RGB565 pixel data to RGB Image Conversion – Output



What it does:

1. Converts hexadecimal pixel values into a NumPy array.
2. Extracts Red, Green, and Blue components from each RGB565 pixel.
3. Converts RGB565 format to RGB888 format.
4. Arranges the pixels into a 176×144 image.
5. Displays the reconstructed image using Matplotlib.

Result:

Successfully captured an image using the OV7675 camera, transferred the hexadecimal pixel data to Python, and reconstructed the image on the computer.

